

## ЛАБОРАТОРНА РОБОТА №5.

### УРАЗЛИВОСТІ І ЗАХИСТ

Мета роботи: Сформувати вміння виявляти та відтворювати типові уразливості вебзастосунків у контрольованому навчальному середовищі, виконувати базове виправлення на рівні коду та конфігурації, а також перевіряти ефективність захисту за принципом «проблема → відтворення → виправлення → перевірка». У результаті студент має навчитися застосовувати елементарні практики безпечної розробки для запобігання SQL Injection, XSS, Broken Access Control (IDOR) та помилкам конфігурації безпеки.

### Теоретичні відомості

#### **Вступ: навіщо безпека у веброзробці**

У попередніх лабораторних було показано, як будувати фронтенд, писати бекенд на Express, інтегрувати fetch() і працювати з SQLite. У даній роботі додається ще один обов'язковий вимір: безпека як властивість системи, а не «післядумка».

#### ***Базові терміни: уразливість, загроза, ризик***

- Уразливість (vulnerability) – дефект у логіці, коді або конфігурації, який можна використати для небажаної поведінки.
  - Приклад (узагальнено): «дані користувача потрапляють у SQL як частина тексту запиту» або «текст користувача вставляється у DOM як HTML».
- Загроза (threat) – потенційний сценарій, що може завдати шкоди, якщо уразливість буде експлуатована (наприклад, «отримати доступ до чужих даних», «виконати скрипт у браузері жертви»).
- Ризик (risk) – оцінка ймовірності реалізації загрози та масштабу наслідків.

Практично: одна й та сама уразливість може мати різний ризик залежно від того, які дані зберігає система і хто має до неї доступ.

Ключова думка: треба навчитись бачити причинно-наслідковий ланцюжок, а не просто «виправляти помилки».

***«Довіряй, але перевіряй»: чому фронтенд не гарантує безпеку***

Фронтенд працює на боці користувача. Це означає:

- користувач може змінити JS-код у DevTools;
- користувач може відправити запит напряму (Postman/curl), минаючи UI;
- користувач може підставити довільні значення в URL/тіло запиту/заголовки.

Тому будь-які «обмеження» на фронтенді (наприклад, приховали кнопку, заборонили поле, зробили disabled) – це покращення UX, але не контроль доступу.

Звідси правило:

- вхідні дані завжди вважаються недовіреними;
- критичні перевірки мають бути на бекенді (особливо доступ до ресурсів і робота з БД).

***Робоча модель лабораторної роботи №5: «атака → наслідки → контрзаходи»***

Для кожного сценарію використовується одна й та сама структура мислення:

1. Поверхня атаки: де система приймає дані (параметри URL, JSON body, заголовки, дані з БД, що потім відображаються).
2. Уразливість: що саме зроблено небезпечно (конкатенація SQL; небезпечне вставляння в DOM; відсутність перевірки власника ресурсу; невірні заголовки/помилки).
3. Експлуатація (відтворення): який ввід/запит демонструє проблему.
4. Наслідки: що з цього може отримати зловмисник (витік, модифікація, виконання JS, доступ до чужих даних).

5. Контрзаходи: що змінюємо (параметри в SQL; безпечний рендер; server-side authorization; hardening конфігурації).
6. Перевірка виправлення: «той самий» сценарій більше не працює, а легітимний функціонал працює.

Ця модель потрібна, щоб виконання лабораторної роботи не перетворилось на набір випадкових «патчів»: треба навчитись пояснювати, чому виправлення правильне і як перевірено результат.

### **Правила виконання лабораторної (етика і безпечне середовище)**

Лабораторна робота № 5 передбачає відтворення уразливостей. Це корисно для навчання, але вимагає чітких обмежень, щоб робота була етичною, безпечною та відтворюваною.

#### ***Де дозволено тестувати***

Дозволені середовища:

- локально на власному ПК (localhost);
- навчальний стенд, якщо його явно надано викладачем і він призначений саме для цієї лабораторної;
- окрема тестова копія власного навчального проекту (не бойова, не публічна).

#### **Заборонено:**

- тестувати будь-які сторонні сайти/сервіси, у тому числі «просто спробувати»;
- тестувати реальні production-системи, навіть якщо до них є доступ.

Це принципова вимога: у цій роботі студенти вчаться захищати – не «атакувати інтернет».

#### ***Які дії вважаються прийнятними в рамках лабораторної***

У лабораторній допускається:

- вводити «проблемний» ввід у власні форми (наприклад, у поле коментаря чи пошуку) для демонстрації XSS/SQLi;

- відправляти запити до власного бекенду (Postman/curl/requests.http) для відтворення IDOR;
- змінювати конфігурацію серверу (помилки, заголовки, CORS) у межах проекту.

**Не допускається:**

- сканування мережі, підбір паролів, масові перебори, навантажувальні атаки;
- будь-які дії, що виходять за межі демонстрації конкретного сценарію, описаного в завданні.

Практичне правило: у кожному сценарії має бути чітка мета і контрольований приклад, а не «пограюся, що вийде».

***Мінімальні правила безпечної організації середовища***

Щоб лабораторна не створила небажаних наслідків:

- Необхідно використовувати тестову базу SQLite (наприклад, dev.db або lab5.db), а не «єдину базу на всі роботи».
- Якщо у проекті є «seed» дані – обов’язково використовувати вигадані (не реальні персональні дані).
- Якщо запити логуються – в логах секрети (ключі, токени) не зберігаються.
- Під час демонстрації XSS не треба зберігати у «payload» нічого, що може викрасти реальні дані. Для навчання достатньо мінімальних маркерів (наприклад, зміна DOM/вивід повідомлення).

Це потрібно, щоб лабораторна залишалась навчальною і безпечною.

***Вимога відтворюваності: що саме треба зафіксувати у звіті***

Для кожного сценарію у звіті має бути 4 блоки:

1. «Було (уразливо)»
  - де саме проблема (маршрут/файл/функція/фрагмент UI);
  - коротко: «яка помилка в підході».
2. «Відтворення»
  - конкретний крок або запит, який демонструє проблему;

- що саме спостерігається (результат/відповідь/поведінка UI).
3. «Виправлення»
- що було змінено (не перепис коду, а логіка: параметризація, textContent, перевірка owner, заголовки, приховування dev-деталей).
4. «Перевірка»
- повторення тих самих кроків/запитів і показ, що проблема не відтворюється;
  - окремо: переконатись, що легітимний сценарій працює.

Докази можуть бути будь-які з перелічених:

- скріншот результату в браузері;
- фрагмент відповіді API (JSON);
- короткий лог сервера;
- приклад запиту в Postman/requests.http.

Важливо: «перевірка» – це не «наче виправлено», а контрольне відтворення.

### ***Принцип мінімальної достатності***

У рамках лабораторної роботи не потрібно робити «всю безпеку світу». Достатньо:

- 3–4 сценарії з переліку;
- локальна демонстрація;
- чіткий «до/після».

Мета лабораторної – сформуванню звичку мислити через «вразливість → контрзахід → перевірка».

### **Untrusted input та моделі захисту (allowlist/blocklist)**

У лабораторній роботі №5 усі сценарії (SQLi, XSS, IDOR, misconfiguration) об'єднує одна спільна ідея: вхідні дані не можна вважати «чесними». Навіть якщо дані приходять із свого ж фронтенду – для бекенду це все одно зовнішній світ.

### ***Поняття «untrusted input» (недовірені дані)***

Недовірені дані – це будь-які значення, які система отримує «ззовні» і не контролює повністю. У даному стеку типовими джерелами є:

- URL параметри: /posts/:id
- query string: /posts?search=...
- тіло запиту (JSON): POST/PUT з даними форми
- HTTP-заголовки: у цій роботі це особливо важливо через X-Demo-UserId
- дані з БД, які колись були введені користувачем (для XSS це критично: «збережений» XSS приходить із БД назад у UI)

Практичний висновок:

- «прийшло від користувача» – означає «може містити що завгодно»;
- навіть «прийшло з БД» може бути небезпечним, якщо це колись записав користувач і потім це відображається як HTML.

### ***Валідація, санітизація, екранування – не одне й те саме***

У побуті ці слова плутають, але в контексті безпеки вони мають різні ролі:

- Валідація (validation) – перевірка, що дані відповідають очікуваному формату/діапазону.
  - Приклад: id – це UUID; rating – число від 1 до 5.
- Санітизація (sanitization) – «очищення» або перетворення даних для безпечного використання (наприклад, прибрати небажані символи/теги).
  - Важливо: санітизація часто має сенс лише в конкретному контексті і не є універсальною «панацеєю».
- Екранування (escaping / output encoding) – безпечне перетворення даних на виході під конкретний контекст (HTML/атрибут/JS/URL), щоб вони не «ламали» інтерпретацію.
  - Для XSS базове правило: екранування/безпечний рендер на виході важливіше, ніж спроби «почистити все на вході».

У даній лабораторній роботі:

- проти SQLi ключове – параметризація (а не санітизація рядка);

- проти XSS ключове – безпечне відображення (а не «видалити всі підозрілі символи»).

### ***Allowlist vs Blocklist (whitelist/blacklist)***

Це дві моделі фільтрації/контролю:

- Allowlist (whitelist): дозволяємо тільки те, що явно дозволено.
  - Наприклад: sort може бути тільки date|title|rating, і все інше – помилка 400.
- Blocklist (blacklist): забороняємо те, що явно заборонено/відомо як погане.
  - Наприклад: «заборонити <script>».

Чому в безпеці частіше рекомендують allowlist / whitelist:

- злі ввідні дані мають майже безмежну кількість форм;
- блоклист легко обійти варіаціями (регістр, пробіли, кодування, інші конструкції).

Практичні рекомендації для студентських проектів:

- allowlist / whitelist використовується для строгих полів (id, числа, enum-значення, напрямок сортування);
- blocklist / blacklist використовується обережно і тільки як додатковий бар'єр, але не як єдиний захист від XSS/SQLi.

***Контекстність: одні й ті самі дані можуть бути «безпечні» в одному місці і небезпечні в іншому***

Один і той самий рядок може бути:

- безпечним як текст у UI;
- небезпечним, якщо його вставити як HTML;
- небезпечним, якщо його підставити в SQL-рядок як частину коду;
- небезпечним, якщо включити в лог/повідомлення так, що воно «розкриває» внутрішні деталі.

Звідси базовий принцип:

- не існує універсального «очищення рядка» один раз і назавжди;

- безпека залежить від того, як саме використовуються дані (SQL, HTML, URL, заголовки, файли).

### ***Мінімальна практична політика, що вважати «правильним»***

У межах лабораторної роботи достатньо дотриматися правил:

- Валідація формату там, де формат відомий
  - id – UUID/число; page – ціле  $\geq 1$ ; sort – з allowlist.
- SQL – тільки через параметри, дані не повинні ставати частиною SQL-коду.
- Дані користувача в DOM – тільки як текст
- Не вставляти недовірені дані в innerHTML.

### **SQL Injection: механіка, типові помилки, виправлення**

У попередніх роботах студентам свідомо рекомендувалось працювати з SQLite через «сирі» SQL-рядки. У даній роботі це буде використано це як навчальний матеріал: той самий підхід, який зручний для швидкого старту, є головним джерелом SQL Injection, якщо в запит потрапляють недовірені дані.

### ***Що таке SQL Injection і чому він виникає***

SQL Injection (SQLi) виникає тоді, коли програміст:

1. бере ввід користувача (query/body/params),
2. вставляє його в SQL-рядок через конкатенацію або шаблонний рядок,
3. і передає цей рядок у БД як «готову команду».

Проблема в тому, що база даних інтерпретує отриманий рядок як код SQL, і частина «даних» може почати працювати як оператори/умови.

Критична ознака: користувацький ввід впливає на структуру SQL-запиту, а не тільки на значення параметрів.

### ***Типові уразливі місця в студентських проектах***

У стеку «Express + SQLite» найчастіше «стріляють» такі місця:

- Пошук / фільтр
  - GET /posts?search=...



- LIKE '%...%' з конкатенацією
- Фільтрація за полем/значенням
  - GET /items?status=...
  - WHERE status = '\${status}'
- Деталі по id
  - GET /notes/:id (особливо якщо id – рядок)
- Сортування / вибір колонки
  - ORDER BY \${sort}
  - Це окремий ризик: параметризація не працює для назв колонок/напрямку сортування, тут потрібен allowlist.

### ***Мінімальна демонстрація проблеми (локально)***

Для лабораторної достатньо показати, що «поганий ввід»:

- або ламає SQL-синтаксис (помилка запиту),
- або змінює логіку фільтрації (повертає не те, що повинно).

### ***Як виправляти правильно: параметризовані запити***

Параметризація означає, що SQL-команда і значення даних передаються окремо:

```
... WHERE title LIKE ?
```

з параметром: "%userInput%"

Тоді драйвер SQLite:

- не підставляє параметр як SQL-код,
- а передає його як дані (значення), які не можуть змінити структуру запиту.

Приклад «було → стало» (sqlite3)

Було (уразливо):

```
const q = req.query.search ?? "";
const sql = `SELECT * FROM Posts WHERE title LIKE '%${q}%'`;
db.all(sql, (err, rows) => res.json(rows));
```

Стало (параметризовано):

```
const q = req.query.search ?? "";
const sql = `SELECT * FROM Posts WHERE title LIKE ?`;
db.all(sql, [`%${q}%`], (err, rows) => res.json(rows));
```

Важливо: не «екранувати лапки» вручну, а саме перейти на параметри.

**Окремий випадок: *ORDER BY* і назви колонок (тут параметри не допоможуть)**

Параметри працюють для значень, але не для ідентифікаторів (назв таблиць/колонок) і не для шматків синтаксису.

Небезпечний патерн:

```
const sort = req.query.sort;
const sql = `SELECT * FROM Posts ORDER BY ${sort}`; // ризик
```

Правильний мінімум – allowlist:

```
const sort = req.query.sort ?? "date";
const allowed = new Set(["date", "title", "rating"]);
const sortColumn = allowed.has(sort) ? sort : "date";

const sql = `SELECT * FROM Posts ORDER BY ${sortColumn} DESC`;
```

Це приклад, де allowlist є доречним і простим.

**Перевірка, що виправлення працює**

Після переходу на параметризовані запити студент має показати у звіті:

- «поганий ввід», який раніше змінював поведінку/ламає запит, більше не впливає на результат (або сприймається як звичайний текст);
- нормальний пошук/фільтр працює як раніше.

Практична рекомендація для перевірки:

зробити 2–3 контрольні запити: «звичайний ввід», «ввід зі спецсимволами», «крайні випадки (порожній рядок)».

**ORM як варіант виправлення (опційно)**

Якщо студент обирає ORM, важливо розуміти:

- ORM зазвичай генерує параметризовані запити автоматично,
- але небезпека повертається, якщо виконувати «raw SQL» і знову конкатенувати рядок.

Тому необхідно використовувати ORM-запити/фільтри так, щоб значення передавалися як дані, а не як рядок SQL.

## **XSS: відбитий/збережений та виправлення через безпечний рендер**

У попередніх роботах навмисно використовувався `innerHTML` замість `textContent`, щоб на реальному прикладі показати, чому це небезпечно. У цій роботі це стає формальним сценарієм: знайти місце, де дані користувача потрапляють у DOM як HTML → відтворити XSS → виправити → перевірити.

### ***Що таке XSS і де він «живе» у вашому стеку***

XSS (Cross-Site Scripting) – це ситуація, коли браузер виконує (або інтерпретує як HTML/JS) те, що насправді є даними, введеними користувачем.

У стеку, який застосовується в лабораторних роботах, XSS майже завжди з'являється так:

1. користувач вводить текст (коментар/назва/опис);
2. бекенд зберігає це в SQLite (або віддає відразу);
3. фронтенд рендерить ці дані через `innerHTML` або подібний «небезпечний» шлях;
4. браузер сприймає частину рядка як HTML/JS.

Ключова ознака: «дані» та «розмітка» змішані в одному каналі.

### ***Види XSS***

Для 1 курсу достатньо двох найпоширеніших типів:

#### ***Відбитий XSS (Reflected)***

Дані приходять у відповідь одразу в рамках одного запиту (наприклад, береться `?q=` і відображається на сторінці як частина HTML).

Типово: «пошук» або «повідомлення про помилку», які відображають введений рядок.

#### ***Збережений XSS (Stored)***

Дані зберігаються (в SQLite) і потім показуються іншим користувачам (або тому ж, хто створив, при наступному відкритті сторінки).

Типово: коментарі, пости, нотатки, тікети.

Для лабораторної зручніше брати `stored XSS`, бо він природно прив'язується до CRUD і БД.

### ***Мінімальне відтворення XSS (локально)***

У звіті має бути показано, що:

- введені «спеціальні конструкції» не відображаються як звичайний текст,
- а спричиняють небажану поведінку в UI (наприклад, зміну структури DOM або виконання небезпечного вмісту).

Для цього необхідно ввести рядок, який містить HTML-теги або конструкції, що можуть змінити DOM при вставці через `innerHTML`.

### ***Чому `innerHTML` небезпечний саме в цьому сценарії***

`innerHTML` говорить браузеру: «ось тобі HTML, розбери його як розмітку».

Якщо підставити туди текст користувача, то браузер може:

- створити нові теги;
- підставити атрибути;
- змінити структуру сторінки.

Тобто рядок перестає бути «рядком» і стає «інструкцією».

### ***Правильне виправлення: безпечне відображення (`output encoding / safe rendering`)***

Найпростіший і правильний шлях: не інтерпретувати дані як HTML.

### ***Варіант А (рекомендований): `textContent` + `DOM API`***

Було (ризиковано):

```
list.innerHTML += `<li>${comment.text}</li>`;
```

Стало (безпечніше):

```
const li = document.createElement("li");
li.textContent = comment.text;
list.appendChild(li);
```

Якщо елемент складніший (кілька полів):

```
const card = document.createElement("div");

const title = document.createElement("h3");
title.textContent = post.title;

const body = document.createElement("p");
body.textContent = post.body;
```

```
card.append(title, body);
container.appendChild(card);
```

Це не так зручно, як було, але це правильно: дані стають текстом, а не розміткою.

### ***Варіант Б: «шаблон» без innerHTML (через createElement)***

Шаблони можна робити, але поля з даними користувача мають виставлятися після створення елемента, через `textContent`.

Тобто:

1. створити «каркас» DOM-елементів;
2. заповнити текстові вузли через `textContent`.

### ***Чому «очищати на вході» – не основний захист від XSS***

Типова помилка: «давайте видалимо `< i >`». Це:

- ламає легітимний текст (наприклад, математичні записи);
- не гарантує захисту (XSS може мати варіанти через різні контексти).

Тому головний захист – безпечне відображення, бо саме воно визначає, чи інтерпретується текст як код.

Валідацію/санітизацію можна залишити як додатковий шар, але не як єдину «панацею».

### ***Перевірка виправлення***

Після змін студент має показати:

- той самий «поганий ввід» тепер відображається як звичайний текст або не впливає на DOM;
- рендер списку/карток не зламався: нормальні коментарі/пости відображаються коректно.

У звіті бажано:

- один скріншот «до» (аномальна поведінка),
- один скріншот «після» (текст як текст),
- коротке пояснення: «замінили innerHTML на DOM API + `textContent`».

## **Broken Access Control / IDOR: суть, демо-ідентифікація, перевірка доступу**

У цій роботі Broken Access Control розглядається на найпростішому, але дуже типовому випадку: IDOR (Insecure Direct Object Reference). Це ситуація, коли користувач отримує доступ до чужого ресурсу, просто підставивши інший id у запиті.

### ***Автентифікація vs авторизація (мінімально необхідне)***

- Автентифікація відповідає на питання: «Хто ти?» (вхід у систему).
- Авторизація відповідає на питання: «Що тобі дозволено?» (права доступу).

IDOR – це помилка авторизації: система «знає», хто користувач, але не перевіряє, чи має він право доступу до конкретного ресурсу.

Для лабораторної достатньо мати стабільний `currentUserId`. Повноцінний логін/пароль – не обов’язковий.

### ***Демо-ідентифікація через X-Demo-UserId (навчальний компроміс)***

Щоб не ускладнювати лабораторну роботу повною системою логіну, використовується демо-ідентифікація:

1. клієнт передає заголовок: `X-Demo-UserId: <id>`
2. сервер читає його та встановлює `currentUserId` у контекст запиту

Це дозволяє фокусуватися на перевірці прав доступу, а не на механіці login/JWT.

Вимоги до поведінки:

- немає `X-Demo-UserId` → 401 Unauthorized;
- `userId` невалідний або користувача не існує → 401 Unauthorized (або 400, в залежності від реалізації).

### ***Мінімальна реалізація демо-ідентифікації (заготовка)***

Нижче – приклад middleware-структури.

```
function demoAuth(req, res, next) {
  const userId = req.header("X-Demo-UserId");
  if (!userId) {
    return res.status(401).json({ error: "Unauthorized" });
  }
}
```

```
// TODO: валідація формату (number або UUID)
// TODO: перевірка, що такий користувач існує (Users у SQLite або seed)

req.user = { id: userId };
next();
}
```

Рекомендація для лабораторної: тримати Users максимально простими (2–3 записи).

### ***Що саме є «ресурсом» для IDOR у навчальному проекті***

Для IDOR потрібен ресурс, який належить користувачу, тобто має поле власника, наприклад, ownerId (або userId, але краще називати явно)

Найзручніше брати сутності, які природно «персональні»:

- PersonalNotes,
- Tickets,
- AccessRequests,
- CheckoutRequests,
- Registrations,
- тощо.

Треба обрати одну сутність, для якої логічно, що записи мають власника. Додати до неї поле ownerId і забезпечите, що всі чутливі операції перевіряють власника.

### ***Як виникає IDOR (типовий вразливий патерн)***

Найпоширеніша помилка: робити доступ «за id» без перевірки власника.

```
app.get("/notes/:id", demoAuth, (req, res) => {
  db.get(`SELECT * FROM Notes WHERE id = ?`, [req.params.id], (err, note) => {
    res.json(note);
  });
});
```

У такій версії будь-який користувач, знаючи id, може отримати чужі дані.

### ***Правильний мінімальний захист: серверна перевірка власника***

Основне правило:

- власник ресурсу визначається сервером, а не клієнтом;

- при читанні/зміні/видаленні сервер перевіряє `ownerUserId == currentUserId`.

Найпростіший і надійний спосіб – «фільтрація в запиті до БД»:

```
app.get("/notes/:id", demoAuth, (req, res) => {
  const sql = `SELECT * FROM Notes WHERE id = ? AND ownerUserId = ?`;
  db.get(sql, [req.params.id, req.user.id], (err, note) => {
    if (!note) return res.status(404).json({ error: "Not found" });
    res.json(note);
  });
});
```

Цей підхід:

- короткий;
- не потребує «другого запиту»;
- одразу гарантує, що неможливо працювати з «чужим» ресурсом у коді.

### ***Які операції треба закрити перевітками***

Для обраного ресурсу рекомендується закрити:

- GET /resource/:id (читання)
- PUT /resource/:id або PATCH (оновлення)
- DELETE /resource/:id (видалення)

Важливо: якщо закрити лише «читання», але не закрити «оновлення» – уразливість частково лишається.

### ***Політика відповіді: 403 чи 404***

При спробі доступу до «чужого» ресурсу повертають один з двох кодів статусу:

403 Forbidden – «ресурс існує, але у даного користувача немає доступу».

404 Not Found – «такого ресурсу не існує» (навіть якщо насправді він існує).

Для студентського проекту простіше часто використовувати 404, бо тоді одна гілка логіки («не знайдено») покриває і «не існує», і «чужий».

Головне: описати у звіті, який варіант обрано і чому.

### ***Перевірка виправлення (що має бути в звіті)***

Для IDOR у звіті має бути:



- приклад доступу до «чужого» ресурсу до виправлення (успішно);
- той самий запит після виправлення (отримання 403/404);
- приклад запиту до «свого» ресурсу після виправлення (успішно).  
Технічно це зручно робити Postman/requests.http, змінюючи лише:
- X-Demo-UserId
- :id у URL

### **Security Misconfiguration: мінімальний hardening конфігурації**

У попередніх роботах студенти вже налаштовували базові речі на бекенді (коди стану, централізовані помилки) та торкалися CORS при інтеграції фронтенд ↔ бекенд. Тепер треба подивитись на це з позиції безпеки: навіть правильний бізнес-код може бути небезпечним через конфігурацію та «дрібні» налаштування.

#### ***Що take Security Misconfiguration у практичному сенсі***

Security Misconfiguration – це коли система:

- розкриває зайву інформацію (dev-деталі, стек-трейси, внутрішні повідомлення);
- має надто «широкі» дозволи (наприклад, CORS для всіх);
- не має базових захисних налаштувань на рівні HTTP-відповіді;
- поводить не послідовно з помилками (інколи 500 з деталями, інколи 200 з "error": true).

У рамках цієї роботи достатньо мінімального hardening'у, який можна застосувати майже в будь-якому Express-проекті.

#### ***Не розкривати dev-деталі в помилках***

Ризик: якщо сервер повертає стек-трейс або внутрішні тексти помилок, це:

- підказує структуру проекту, маршрути, назви таблиць/полів;
- спрощує пошук вразливих місць.

Вимога: у production-режимі клієнт не повинен бачити stack trace.

Мінімальна модель:

- сервер логуватиме деталі (у консоль/файл) – це ок;
- клієнту віддає «чисте» повідомлення.

Приклад коду:

```
app.use((err, req, res, next) => {
  console.error(err);

  const isDev = process.env.NODE_ENV !== "production";
  res.status(500).json({
    error: "Internal Server Error",
    details: isDev ? String(err.message ?? err) : undefined
  });
});
```

Правило: один формат помилки + жодних внутрішніх деталей назовні в production.

### ***Коректні повідомлення про помилки і статус-коди***

Ціль – щоб клієнт бачив:

- 4xx для помилок користувача (невалідні дані, неавторизований доступ);
- 5xx для помилок сервера (винятки, збої підключення до БД).

Це важливо з двох причин:

- при демонстрації сценаріїв (SQLi/XSS/IDOR) треба чітко показати, що система правильно реагує;
- «красиві» помилки без деталей – частина hardening.

У звіті достатньо 2–3 прикладів:

- 401 коли немає X-Demo-UserId;
- 404/403 при IDOR після виправлення;
- 400 при невалідному форматі.

### ***Базові безпечні HTTP-заголовки (мінімум)***

У цій роботі не потрібно глибоко занурюватися в CSP/advanced security headers. Достатньо 2–3 заголовків, які:

- не ламають локальну розробку,
- легко перевіряються в DevTools/Network або через curl.

Рекомендований мінімум:

- X-Content-Type-Options: nosniff

- Захищає від «піднюхування» типів контенту браузером у деяких сценаріях.
- X-Frame-Options: DENY
  - Зменшує ризики clickjacking (вбудовування в iframe).
- Referrer-Policy: no-referrer (або більш м'яка політика)
  - Контроль, що саме браузер відправляє в Referer.

Заготовка middleware:

```
app.use((req, res, next) => {
  res.setHeader("X-Content-Type-Options", "nosniff");
  res.setHeader("X-Frame-Options", "DENY");
  res.setHeader("Referrer-Policy", "no-referrer");
  next();
});
```

Перевірка у звіті: показати фрагмент відповіді (headers) з DevTools або curl -I.

### ***CORS як частина misconfiguration***

Студенти вже налаштували CORS, тому тут лише принцип безпеки:

«Access-Control-Allow-Origin: \*» – це нормально для публічного API без куки/секретів, але для навчального проекту з персональними ресурсами це часто надмірно широко.

Мінімально правильний підхід: дозволяти тільки origin фронтенду (локально це зазвичай http://localhost:<порт>).

CORS має бути обмежений конкретним origin, який використовується для фронтенду, якщо бекенд віддає персональні дані або має операції зміни стану.

### ***Вимкнення «зайвих» dev-можливостей (мінімально)***

Під «dev-деталлями» в контексті Express найчастіше мається на увазі:

- детальні stack trace у відповіді;
- занадто докладні повідомлення від middleware;
- «verbose» режим логів у відповідях клієнту.

Як підсумок: логувати можна детально, але відповідь клієнту має бути стриманою.

### ***Що має бути в звіті для цього сценарію***

Для сценарію Misconfiguration у звіті достатньо:

- приклад помилки «до» (якщо були stack trace/деталі у відповіді) і «після» (чисте повідомлення);
- підтвердження заголовків (скрін/curl -I);
- коротка примітка про CORS-політику (який origin дозволений).

### **Перевірка виправлень і регресія**

В останній лабораторній роботі важливо не тільки «виправити», а довести, що:

- уразливість справді зникла;
- легітимний функціонал не зламався;
- виправлення не є випадковим (тобто зрозуміло, чому воно працює).

### ***Принцип «до/після» як обов'язковий доказ***

Для кожного сценарію (SQLi, XSS, IDOR, misconfiguration) у звіті має бути пара:

1. До виправлення: конкретний крок/запит → конкретний небажаний результат.
2. Після виправлення: той самий крок/запит → очікувано безпечний результат.

Важливо: «до» і «після» мають відрізнятися мінімально (ідеально – тільки кодом у репозиторії). Тоді перевірка є чесною і відтворюваною.

### ***Мінімальні артефакти перевірки***

Достатньо одного з варіантів для кожного сценарію (або комбінації):

- API-запит (Postman/Insomnia/requests.http) + відповідь JSON
- скріншот з браузера (Network/Console або результат рендеру)
- короткий лог серверу (1–3 рядки), якщо він потрібен для пояснення

Рекомендація для студентів: обрати єдиний формат для всіх сценаріїв (наприклад, «запит + відповідь»), щоб звіт був компактним.

## ***Перевірка по сценаріях: що вважати достатнім***

### ***SQL Injection***

Мінімум:

- один «нормальний» запит (функціонал працює);
- один «проблемний» ввід, який до виправлення змінював поведінку/давав помилку SQL, а після виправлення:
  - або сприймається як текст і не впливає на структуру SQL,
  - або коректно обробляється без аварій.

Критерій успіху: дані не можуть впливати на структуру SQL-запиту, бо йдуть параметром.

### ***XSS***

Мінімум:

- до виправлення: ввід із HTML-конструкціями призводив до зміни DOM/аномальної поведінки;
- після виправлення: той самий ввід відображається як текст або без впливу на DOM.

Критерій успіху: дані користувача більше не потрапляють у небезпечні канали.

### ***IDOR***

Мінімум:

- до виправлення: підставляння чужого id (при іншому X-Demo-UserId) дозволяє отримати/змінити чужий ресурс;
- після виправлення: той самий запит повертає 403 або 404;
- при цьому «свій» ресурс доступний і «до», «і після».

Критерій успіху: перевірка власника/прав відбувається на бекенді, а не у фронтенді.

### ***Misconfiguration***

Мінімум:

- підтвердити наявність 2–3 заголовків у відповіді;

- показати, що помилки не розкривають dev-деталі в «production-режимі»;
  - коротко описати політику CORS (обмежений origin).
- Критерій успіху: клієнт бачить стримані помилки й базові захисні заголовки.

### ***Типові помилки під час перевірки***

- «Перевіряти тільки один endpoint»: IDOR/SQLi можуть лишитися в іншому місці.
- «Виправляти XSS у UI, але лишати небезпечний innerHTML у іншій функції рендера.»
- «Повертати 500 замість 401/403/404»: у звіті це виглядає як відсутність контролю помилок.
- «Показувати тільки ‘після’, без ‘до’»: тоді незрозуміло, що саме було виправлено.

## **Практична частина**

### **Загальні вимоги (для всіх рівнів)**

- Виконано в навчальному середовищі (локально/навчальний стенд).
- Для кожного обраного сценарію присутня структура: «було → відтворення → виправлення → перевірка».
- Для сценарію IDOR має існувати поняття поточного користувача (демо або повна автентифікація).

### ***Формат здачі (для всіх рівнів)***

1. Репозиторій з кодом.
2. Короткий звіт (1–3 сторінки або REPORT.md), де по кожному сценарію є:
  - де саме в проєкті відтворюється проблема (ендпойнт/форма/компонент);
  - як проявляється (що саме стало «не так»);
  - що змінено для виправлення (коротко);

- як перевірено (скрін/лог/запит + очікуваний результат).

### ***Сценарій***

#### ***Сценарій А – SQL Injection (SQLi)***

Відтворення: у проєкті є запит до SQLite, який будується через конкатенацію рядка (наприклад, пошук/фільтр/авторизація/деталі за параметром).

Виправлення: замінити на параметризований запит або використати ORM, де параметризація виконується коректно.

Перевірка: показати, що «шкідливий ввід» більше не змінює логіку запиту (результати коректні, помилок немає).

#### ***Сценарій Б – XSS (відбитий або збережений)***

Відтворення: дані від користувача відображаються на сторінці так, що можуть інтерпретуватися як HTML/скрипт (наприклад, коментарі/опис/назва).

Виправлення:

на фронтенді: безпечне відображення (екранування, `textContent` замість небезпечних вставок у DOM);

за потреби: базова валідація на бекенді (відсікання явно небезпечних конструкцій або allowlist).

Перевірка: показати, що після виправлення введений «HTML/JS-ввід» відображається як текст або блокується.

#### ***Сценарій В – Broken Access Control / IDOR***

Відтворення: є ресурс, який «належить користувачу» (наприклад, нотатка, заявка, пост, коментар, реєстрація), але доступ можна отримати, підставивши чужий id у URL/запиті.

Виправлення: на бекенді додати перевірку власника/прав доступу (ідентифікатор користувача брати з сесії/токена, а не з тіла запиту).

Перевірка: чужий ресурс більше не читається/не змінюється (коректний 403 або 404), свій – працює.

### ***Сценарій Г – Security Misconfiguration (мінімальний харднінг)***

Виправлення/налаштування (мінімум):

прибрати «dev-деталі» з помилок (без стек-трейсів/внутрішніх деталей назовні);

коректні повідомлення про помилки + коди стану;

базові безпечні заголовки (мінімум 2–3 доречні);

(за можливості) більш строгий CORS (не «дозволити всім» без потреби).

Перевірка: показати заголовки/поведінку в відповіді та приклад помилки «після виправлення».

### **Порядок виконання роботи**

1. Ознайомитись з теоретичним матеріалом.
2. Виконати індивідуальні завдання.
3. Завантажити код у той же репозиторій, куди був завантажений код лабораторної роботи №1. Остаточну версію позначити тегом «1.0.0».
4. Підготувати звіт по виконаній роботі, оформлений згідно з ДСТУ 3008:2015. У звіті вказати посилання на репозиторій з п.3.

### **Вимоги за рівнями оцінювання**

#### ***Виконання «на задовільно»***

- Зробити 2 сценарії:
  - SQLi (A) – відтворення + виправлення параметризацією/ORM + перевірка.
  - IDOR (B) – відтворення + серверна перевірка доступу + перевірка.
- Мінімальні вимоги:
  - у звіті по кожному сценарію є «до/після»;
  - виправлення не ламає базовий функціонал (CRUD/пошук тощо);
  - обробка помилок не «падає» з 500 там, де можна повернути 4xx.
  - Реалізовано спрощене визначення currentUserId через заголовок X-Demo-UserId: <id>.



- Узгоджена поведінка бекенду:
  - немає заголовка → 401
  - невалідний/невідомий користувач → 401 (або 400, але один підхід всюди)
- У сутності, обраній для IDOR, є ownerId (або еквівалент), і перевірка доступу виконується на бекенді.

### ***Виконання «на добре»***

- Зробити 3 сценарії: А + В + (Б або Г).
- Додаткові вимоги:
  - для XSS: виправлення виконано саме як безпечне відображення даних (а не «заборонили все підряд»), і описано, чому так;
  - або для Misconfiguration: додані базові заголовки + прибрані dev-деталі + коди помилок узгоджені;
  - є централізована обробка помилок (щоб відповіді були однакові за форматом);
  - у звіті є коротка таблиця «ризик → наслідок → виправлення» (по 1–2 речення на пункт).

### ***Виконання «на відмінно»***

- Зробити всі 4 сценарії: А + Б + В + Г.
- Додаткові вимоги:
  - бекенд реалізовано на TypeScript;
  - для SQLi: використано параметризацію або ORM так, щоб це було видно в коді та поясненні;
  - для Access Control: перевірка прав доступу зроблена на кожній чутливій операції (read/update/delete) для обраного ресурсу;
  - для Misconfiguration: додано набір безпечних заголовків і показано, що вони реально присутні у відповіді;

### Додаткові завдання (за бонусні бали)

1. Суть: реалізувати нормальну автентифікацію та використовувати її замість X-Demo-UserId для визначення currentUserId.
  - Є POST /auth/login (і register або seed-користувачі).
  - Паролі зберігаються не у відкритому вигляді (хеш + сіль).
  - Після логіну клієнт отримує стан автентифікації:
    - ◆ cookie session або JWT (будь-який один варіант).
  - Захищені ендпойнти визначають currentUserId з автентифікації, а не з X-Demo-UserId.
  - У сценарії IDOR показано «до/після» вже з повною автентифікацією (тобто X-Demo-UserId більше не потрібний).
  - Поведінка 401/403/404 узгоджена та послідовна.
  - Обмеження повторних спроб логіну (простий rate-limit або затримка).
  - Коректні повідомлення про помилки без розкриття деталей («невірні облікові дані» без уточнень).
  - logout / інвалідація сесії;
  - базові ролі (user/admin) і обмеження доступу до адмін-ендпойнтів;
  - мінімальний захист від brute force (наприклад, ліміт спроб логіну) – якщо не складно.
  - Примітка: якщо реалізовано повну автентифікацію, допускається не реалізовувати X-Demo-UserId взагалі – за умови, що IDOR-сценарій має чіткий currentUserId через login/session/JWT.

### Формальні критерії оцінювання

#### *Задовільно*

1. Реалізовано базову ідентифікацію/контекст користувача (наприклад, заголовок X-Demo-UserId або аналог) і повернення 401, якщо контексту немає.

2. Показано і задокументовано SQL Injection у небезпечній версії (мінімальний PoC-запит) і пояснено, чому ін'єкція проходить.
3. Виправлено SQLi: параметризовані запити (або інший механізм) + повторний PoC показує, що ін'єкція не проходить.
4. Показано і задокументовано IDOR (доступ до чужого ресурсу) у небезпечній версії і пояснено, де саме «дірка».
5. Виправлено IDOR: перевірка прав доступу на бекенді, повертається 403, frontend не може «сховати» проблему.

### ***Добре***

1. Додано єдині правила авторизації: перевірка власника/ролі винесена в middleware/функцію, а не копіюється в кожному роуті.
2. Додано захист від XSS у UI: відмова від небезпечного innerHTML там, де дані користувача, або санітизація за необхідності.
3. Є структурований звіт/README: для кожної уразливості «як відтворити → як виправлено → як перевірити, що більше не відтворюється».
4. Усі помилки безпеки повертаються у вашому єдиному форматі помилок (code/message) і це відображається на фронтенді коректно.
5. Налаштовано CORS/headers так, щоб не було відверто небезпечної конфігурації «все можна всім».

### ***Відмінно***

1. Є повний «security regression» набір: короткі curl/HTTP-сценарії для SQLi/IDOR/XSS, які повторювано показують «було/стало».
2. Додано мінімальні security headers (в межах курсу) і пояснено, що вони дають саме у цьому проекті.
3. Реалізовано принцип найменших привілеїв: навіть із валідним контекстом користувача доступ суворо обмежений (нема «зайвих» ендпойнтів/полів).
4. Код організовано: middleware/валідація/доступи читаються і легко перевіряються на рев'ю.

### Контрольні запитання

1. Що таке уразливість, загроза та ризик у контексті вебзастосунків, і як ці поняття пов'язані між собою?
2. Які типові джерела недовірених даних (untrusted input) існують у вебзастосунку на Express (query, params, body, headers), і чому дані з БД також можуть бути недовіреними?
3. У чому відмінність між валідацією, санітизацією та екрануванням, і в яких сценаріях (SQLi/XSS) який підхід є ключовим?
4. Що означають моделі allowlist і blocklist (whitelist/blacklist), і чому allowlist зазвичай надійніший для контролю формату параметрів?
5. За яких умов виникає SQL Injection, і чому побудова SQL через конкатенацію рядків створює уразливість?
6. У чому суть параметризованих запитів, і чому вони запобігають SQL Injection на рівні механіки виконання запиту?
7. Чому параметризація не застосовується безпосередньо до назв колонок і конструкцій на кшталт ORDER BY, і який підхід використовується для безпечної підтримки сортування?
8. Що таке XSS, і в чому концептуальна різниця між reflected XSS та stored XSS?
9. Чому вставка даних користувача через innerHTML (або аналогічні механізми) є небезпечною, і який принцип безпечного рендерингу даних у DOM?
10. Чому «очищення» введення (видалення символів або тегів) не є універсальним захистом від XSS, і чому безпечне відображення на виході вважається базовим контрзаходом?
11. У чому різниця між автентифікацією та авторизацією, і до якого класу проблем належить IDOR?
12. Що таке IDOR (Insecure Direct Object Reference) і чому доступ до ресурсу «лише за id» без перевірки прав доступу вважається уразливістю?

13. Чому «сховати кнопку» або «не показувати чужі записи в UI» – не захист від IDOR? Як можна обійти фронтенд?
14. Які підходи до відмови в доступі (403 vs 404) застосовуються при захисті від IDOR, і в чому їхні практичні відмінності?
15. Що таке Security Misconfiguration у мінімальному розумінні для Express-застосунку, і які типові прояви пов'язані з надмірними dev-деталлями в помилках та некоректними налаштуваннями доступу/відповідей?